



OMNIX QUANTUM LTD

STANDARDS TRACK · OPEN STANDARD · MAY 2026

RFC-ATF-2

Agent Trust Fabric Runtime Governance Continuity

Extension to RFC-ATF-1 · Version 1.0.0

ABSTRACT

RFC-ATF-2 extends the Agent Trust Fabric delegation protocol (RFC-ATF-1, DOI: 10.5281/zenodo.20155016) to cover the full execution lifecycle of long-running autonomous agent workflows. RFC-ATF-1 established cryptographic boundary attestation: proof that an agent possessed a valid, human-originated authority grant at the moment of execution admission. RFC-ATF-2 addresses the Runtime Governance Gap — the structural absence of continuous governability supervision between admission and completion. This extension introduces the Runtime Continuity Record (RCR), the Continuity Eligibility Score (CES), the Authority Fragmentation Guard (AFG), and the Reauthorization Challenge (RC) Protocol. Eight new invariants (RGC-INV-001–008) extend the ATF stack to 14 total formally model-checkable invariants. The reference implementation is currently running in production.

Author	Harold Nunes · Editor
Organization	OMNIX QUANTUM LTD · United Kingdom
Extends	RFC-ATF-1 (DOI: 10.5281/zenodo.20155016)
SSRN Reference	6757339
Signing Algorithm	ML-DSA-65 (Dilithium-3, FIPS 204)
Invariants	14 total (ATF-INV-001–006 + RGC-INV-001–008)
Status	Draft — Zenodo submission in progress
DOI	Pending assignment
Contact	standards@omnixquantum.com

Table of Contents

1. Introduction
2. Problem Statement: The Runtime Governance Gap
3. Conventions and Terminology
4. Architecture: RGC Layer
 - 4.1. Position in the ATF Stack
 - 4.2. Session Lifecycle
 - 4.3. Relationship to TAR
5. Runtime Continuity Record (RCR)
6. Continuity Eligibility Score (CES)
7. Continuity Chain
8. Authority Fragmentation Guard (AFG)
9. Escalation Protocol
10. Reauthorization Challenge (RC)
11. Sampling Strategy
12. RGC Invariants
13. Wire Format
14. Verification Protocol
15. Persistence Schema
16. API Endpoints
17. Security Considerations
18. Compliance Mapping
19. Extension Points
20. Relationship to RFC-ATF-1
21. References
22. Appendix A — CES Computation Examples
23. Appendix B — RGC Compliance Checklist
24. Appendix C — Implementation Notes

Invariants Reference — All 14 ATF Invariants

Invariant	Standard	Property
ATF-INV-001	RFC-ATF-1	Monotonic Authority Reduction (MAR)
ATF-INV-002	RFC-ATF-1	All DRs signed by delegating principal
ATF-INV-003	RFC-ATF-1	Chain root traceability to Tier-1
ATF-INV-004	RFC-ATF-1	No grant exceeds maximum authority budget
ATF-INV-005	RFC-ATF-1	Receipt immutability
ATF-INV-006	RFC-ATF-1	Independent verifiability without platform
RGC-INV-001	RFC-ATF-2	Every RCR anchored to a valid TAR
RGC-INV-002	RFC-ATF-2	CES computed from real-time values only
RGC-INV-003	RFC-ATF-2	HALT terminates execution & revokes sub-tasks
RGC-INV-004	RFC-ATF-2	Aggregate budget $\leq$ AFG limit
RGC-INV-005	RFC-ATF-2	All RCRs PQC-signed and immutable
RGC-INV-006	RFC-ATF-2	Continuity chain is acyclic and monotonic
RGC-INV-007	RFC-ATF-2	CES inputs must meet freshness requirements
RGC-INV-008	RFC-ATF-2	RC TTL enforced – auto-HALT on expiry

ATF-INV-001–006 are proven in TLA+ using the same formal methods methodology as AWS DynamoDB and Azure Cosmos DB.

RGC-INV-001–008 are verified by the reference implementation test suite (82 tests).

Internet-Draft OMNIX QUANTUM LTD

Category: Standards Track H. Nunes, Ed.

ISSN: pending May 2026

RFC-ATF-2: Agent Trust Fabric – Runtime Governance Continuity

Extension to RFC-ATF-1 · Version 1.0.0 · OMNIX QUANTUM Open Standard

Abstract

This document specifies the Runtime Governance Continuity (RGC) extension to the Agent Trust Fabric (ATF) delegation protocol defined in RFC-ATF-1. RFC-ATF-1 established the cryptographic foundation for verifiable agent authority delegation at the point of admission. RFC-ATF-2 extends that foundation to cover the full execution lifecycle of long-running autonomous agent workflows.

The central contribution of this extension is the Runtime Continuity Record (RCR) – a PQC-signed, TAR-anchored authority health artifact emitted at governed intervals throughout agent execution. A sequence of RCRs forms a Continuity Chain: a cryptographic timeline connecting the admission event (TAR) to the execution termination event, with authority health verifiable at every sampled nanosecond in between.

RFC-ATF-2 introduces:

(1) The Continuity Eligibility Score (CES) – a composite, weighted, real-time metric quantifying the runtime health of an agent's authorization across four dimensions: temporal validity, authority budget consumption, contextual fidelity, and integrity signal.

(2) The Authority Fragmentation Guard (AFG) – a protocol enforcing aggregate budget constraints across concurrent sub-agents sharing a delegation chain root, closing an attack vector that individual-level Monotonic Authority Reduction (RFC-ATF-1, §7.1) cannot detect.

(3) The Escalation Protocol – a structured, threshold-triggered response framework that issues signed Continuity Escalation Events (CEEs) and, at CRITICAL threshold, formal Reauthorization Challenges (RCs) requiring Tier-1 response.

(4) Eight new invariants (RGC-INV-001 through RGC-INV-008) that are formally model-checkable and extend the six invariants of RFC-ATF-1.

An implementation that complies with both RFC-ATF-1 and RFC-ATF-2 is designated ATF-RGC-Compliant.

Status of This Memo

This is an OMNIX QUANTUM Open Standard, published under the OMNIX Open Governance License v1.0. This document extends RFC-ATF-1 and MUST be read in conjunction with it. Implementers of RFC-ATF-1 who wish to add runtime continuity monitoring capabilities SHOULD adopt RFC-ATF-2 as specified herein.

Feedback and errata should be submitted to the OMNIX Standards Track at standards@omnixquantum.com.

This document is a product of the OMNIX QUANTUM Standards Working Group. It has been approved for publication by the OMNIX Technical Committee.

DOI: pending (Zenodo submission in progress)

SSRN: pending

Copyright Notice

Copyright (c) 2026 OMNIX QUANTUM LTD. All rights reserved.

This document may be reproduced for implementation purposes.

Acknowledgements

The runtime governance gap formally identified herein was independently recognized by Akhilesh Warik (Runtime AI Governance Research, May 2026) whose public commentary on the architectural separation between boundary attestation and continuous governability supervision directly motivated the formalization of ADR-159 and this extension document.

Table of Contents

1. Introduction
2. Problem Statement: The Runtime Governance Gap
3. Conventions and Terminology
4. Architecture: RGC Layer
 - 4.1. Position in the ATF Stack
 - 4.2. Session Lifecycle
 - 4.3. Relationship to TAR (RFC-ATF-1 Extension: Temporal Authority)
5. Runtime Continuity Record (RCR)
 - 5.1. Record Format
 - 5.2. Identifier Format
 - 5.3. Content Hash Construction
 - 5.4. PQC Signature
 - 5.5. Immutability (RGC-INV-005)
6. Continuity Eligibility Score (CES)

6.1. Formula
6.2. Component Definitions
6.3. Threshold Levels
6.4. Computation Requirements (RGC-INV-002)
7. Continuity Chain
7.1. Chain Structure
7.2. Acyclicity (RGC-INV-006)
7.3. TAR Anchoring (RGC-INV-001)
7.4. Chain Verification
8. Authority Fragmentation Guard (AFG)
8.1. The Fragmentation Attack
8.2. AFG Protocol
8.3. Fragmentation Score
8.4. AFG Invariant (RGC-INV-004)
9. Escalation Protocol
9.1. Continuity Escalation Event (CEE)
9.2. Threshold-to-Action Mapping
9.3. Escalation Event Format
10. Reauthorization Challenge (RC)
10.1. Protocol Overview
10.2. RC Format
10.3. Challenge-Response Sequence
10.4. TTL Enforcement (RGC-INV-008)
10.5. Auto-HALT on Expiry (RGC-INV-003)
11. Sampling Strategy
11.1. Execution Profiles
11.2. Sampling Intervals
11.3. Manual Sampling
12. RGC Invariants
13. Wire Format
13.1. Runtime Continuity Record (JSON)
13.2. Continuity Eligibility Score (JSON)
13.3. Continuity Escalation Event (JSON)
13.4. Reauthorization Challenge (JSON)
13.5. RCR Verification Request/Response
14. Verification Protocol
14.1. Single RCR Verification
14.2. Continuity Chain Verification
14.3. CES Recomputation
14.4. Independent Offline Verification
15. Persistence Schema

15.1. atf_runtime_continuity

15.2. atf_continuity_escalations

16. API Endpoints

17. Security Considerations

17.1. CES Manipulation

17.2. Chain Replay Attack

17.3. Fragmentation Attack

17.4. Clock Drift Attack

17.5. RC Forgery

17.6. Phantom HALT

17.7. Budget Inflation Attack

18. Compliance Mapping

19. Extension Points

20. Relationship to RFC-ATF-1

21. References

22. Appendix A – CES Computation Examples

23. Appendix B – RGC Compliance Checklist

24. Appendix C – Implementation Notes

1. Introduction

RFC-ATF-1 defined a complete protocol for cryptographic agent-to-agent authority delegation. At its core, RFC-ATF-1 enables any party to verify, without platform access, that an autonomous agent possessed a specific, bounded, human-originated grant of authority at the moment its execution was admitted.

This is admission-time governance: a point-in-time proof.

Admission-time governance is necessary but structurally insufficient for long-running agent executions. Consider a workflow that runs for ninety minutes. RFC-ATF-1 (extended by Temporal Authority, ADR-157) provides proof that the delegating authority was valid when the workflow was admitted – second zero. But it cannot answer:

"At minute forty-seven, when the agent committed the transaction – was the authority still valid? Had the budget been depleted by a sibling agent spawned at minute twelve? Had the scope drifted beyond the operational context for which the authority was granted? Were there active anomalies on the delegation chain that should have triggered a restriction?"

These are not theoretical concerns. In multi-agent systems, authority can become invalid mid-execution in four structurally distinct ways:

(a) Temporal expiry – the Delegation Receipt expires before the

execution completes.

(b) Budget exhaustion – the authority budget is consumed, in whole or in part by concurrent sub-agents, below a safe operational level.

(c) Context drift – the operational context for which authority was granted diverges from the actual context of execution, rendering the original scope defensibility void.

(d) Integrity degradation – governance anomalies accumulate on the delegation chain, signaling environmental compromise.

None of these conditions are detectable by RFC-ATF-1 alone.

Governance frameworks that rely solely on admission-time verification leave the ongoing validity of authority as an assumption – trusted implicitly from admission to completion.

RFC-ATF-2 closes this gap.

2. Problem Statement: The Runtime Governance Gap

Define the following terms:

Boundary Attestation (BA):

Cryptographic proof that a delegation grant was valid at time T_0 , the moment of execution admission. RFC-ATF-1 + Temporal Authority (ADR-157) provide full BA.

Continuous Governability Supervision (CGS):

Ongoing verification that the delegation grant remains valid and within safe operational parameters at every moment $T_0 < t < T_f$, where T_f is the execution completion time.

The Runtime Governance Gap is defined as:

$\text{Gap} = \text{CGS} - \text{BA}$

An ATF-1-only deployment has $\text{Gap} > 0$ for any execution with $T_f > T_0$, i.e., any execution with non-zero duration. In practice, for executions of duration d seconds, the window of unverified authority health is exactly d seconds.

For SHORT executions ($d < 60\text{s}$), the gap is operationally tolerable.

For MEDIUM (60s–600s) and LONG ($> 600\text{s}$) executions, and especially for STREAMING (unbounded) executions, the gap represents a structural governance failure under regulatory frameworks requiring continuous monitoring (EU AI Act Art. 9, DORA Art. 9, NIST AI RMF Govern 1.4).

RFC-ATF-2 closes the gap by establishing CGS as a first-class protocol concern.

3. Conventions and Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174].

This document uses all terms defined in RFC-ATF-1 §2 without redefinition. Additional terms introduced by this extension:

Runtime Continuity Record (RCR):

A PQC-signed, immutable authority health snapshot for a specific agent execution at a specific nanosecond. Identified by `ATFRCCR-{16HEX}`.

Continuity Eligibility Score (CES):

A composite real number in `[0.0, 100.0]` representing the runtime health of an agent's authorization at the moment of an RCR. Computed from four weighted components: T (temporal), B (budget), D (context fidelity), I (integrity).

Continuity Chain:

The ordered, acyclic sequence of RCRs for a single execution session, linked via `predecessor_rcr_id`, anchored at one end to the admission TAR and at the other to the terminal RCR.

Continuity Session:

The in-engine tracking state for a single long-running execution, keyed by the admission TAR identifier.

Continuity Escalation Event (CEE):

A PQC-signed artifact emitted when CES crosses a threshold boundary, carrying the threshold crossed, the recommended action, and a response TTL. Identified by `ATFCEE-{16HEX}`.

Reauthorization Challenge (RC):

A PQC-signed formal request for mid-execution authority renewal, issued to the Tier-1 authority when CES drops to CRITICAL. Identified by `ATFRC-{16HEX}`.

Authority Fragmentation Guard (AFG):

A protocol enforcing that the aggregate authority budget across all active Continuity Sessions sharing a `chain_root_id` does not exceed a configurable fraction of the chain root budget.

Fragmentation Score:

The aggregate budget consumption percentage across all active sessions on a chain root, reported in each RCR as a real number

in [0.0, 100.0].

CES Threshold Level:

One of five ordered status values derived from CES:

NOMINAL, MONITORING, WARNING, CRITICAL, HALT.

Temporal Health (T):

The T-component of CES: time remaining on the source DR as a fraction of its total authorized lifetime, expressed as [0, 100].

Budget Health (B):

The B-component of CES: authority budget remaining as a fraction of the budget at admission, expressed as [0, 100].

Context Fidelity (D):

The D-component of CES: 100 minus the context drift percentage reported by the Scope Authorization Engine (ADR-147).

Integrity Score (I):

The I-component of CES: 100 minus ten times the count of active governance anomalies on the delegation chain.

TAR:

Temporal Admissibility Record – the admission anchor for an RGR chain (defined in ATF Extension: Temporal Authority, ADR-157).

AFG Limit:

The maximum fraction of chain root budget permitted across all active sub-delegations. Default: 0.90. Configurable via AFG_FRAGMENTATION_LIMIT environment variable.

4. Architecture: RGC Layer

4.1. Position in the ATF Stack

RFC-ATF-2 adds a fourth operational layer to the three-layer ATF architecture defined in RFC-ATF-1 §3:

Layer 1 – Identity Plane (RFC-ATF-1 §4):

Agent Identity Records.

Layer 2 – Delegation Plane (RFC-ATF-1 §5):

Delegation Receipts and Trust Lattice.

Layer 3 – Verification Plane (RFC-ATF-1 §8):

Independent, offline receipt verification.

Layer 4 – Runtime Continuity Plane (RFC-ATF-2):

Continuous authority health monitoring, escalation, and reauthorization throughout execution lifecycle.

Layer 4 is non-destructive with respect to Layers 1-3: it reads from but does not modify the Delegation Plane. A Layer 4 failure MUST NOT prevent a Layer 3 verification from completing.

4.2. Session Lifecycle

A Continuity Session has the following lifecycle:

ADMITTED:

`start_session()` called by the execution engine, anchored to a validated TAR. Initial CES is computed from real-time inputs.

ACTIVE:

`session.status = "ACTIVE"`. RCRs emitted at governed intervals. CES monitored continuously.

MONITORING / WARNING / CRITICAL:

CES has crossed a threshold. Escalation protocol active.

HALTED:

`session.status = "HALTED"`. Triggered by CES < 10 or RC TTL expiry. Terminal RCR issued. Sibling sessions on the same chain root receive revocation signal.

STOPPED:

`session.status = "STOPPED"`. Normal execution completion. Terminal RCR issued with `sample_reason = "EXECUTION_COMPLETE"`.

REVOKED_BY_HALT:

`session.status = "REVOKED_BY_HALT"`. Set by the engine on sibling sessions when the chain root session enters HALT.

4.3. Relationship to TAR

Every RCR MUST carry a `tar_id` that references a valid TAR in the `atf_temporal_records` table (RGC-INV-001). The TAR provides:

- (a) The precise nanosecond at which execution was admitted.
- (b) The DR validity proof at T_0 .
- (c) The baseline against which temporal health (T-component) is computed.

The continuity chain conceptually begins at the TAR and ends at the terminal RCR. An external verifier can reconstruct the complete execution governance timeline by joining: TAR + ordered RCR chain.

5. Runtime Continuity Record (RCR)

5.1. Record Format

An RCR MUST contain the following fields:

`rcr_id` (string, REQUIRED):

Unique RCR identifier. Format: ATFRCCR-{16HEX}.

MUST be globally unique within the issuing platform.

`tar_id` (string, REQUIRED):

The Temporal Admissibility Record that admitted this execution.

MUST NOT be null (RGC-INV-001).

`delegation_id` (string, REQUIRED):

The Delegation Receipt being monitored. Format: ATFDR-{16HEX}.

`agent_id` (string, REQUIRED):

The executing agent's identity. Format: AID-{DOMAIN}-{16HEX}.

`chain_root_id` (string, REQUIRED):

The human Tier-1 chain root. Identical for all RCRs in a session.

`execution_ns` (integer, REQUIRED):

Nanosecond-precision Unix timestamp of this CES sample.

MUST be strictly greater than the predecessor RCR's `execution_ns`

(RGC-INV-006).

`execution_ts` (string, REQUIRED):

ISO 8601 UTC representation of `execution_ns`.

`ces_score` (float, REQUIRED):

Computed CES at `execution_ns`. Range: [0.0, 100.0].

`ces_temporal` (float, REQUIRED):

T-component. Range: [0.0, 100.0].

`ces_budget` (float, REQUIRED):

B-component. Range: [0.0, 100.0].

`ces_context` (float, REQUIRED):

D-component. Range: [0.0, 100.0].

`ces_integrity` (float, REQUIRED):

I-component. Range: [0.0, 100.0].

`continuity_status` (string, REQUIRED):

One of: NOMINAL, MONITORING, WARNING, CRITICAL, HALT.

Derived from `ces_score` per §6.3.

`predecessor_rcr_id` (string, OPTIONAL):

The preceding RCR in the continuity chain. NULL for the first

RCR in a session.

`budget_at_admission` (float, REQUIRED):

Authority budget as recorded at TAR issuance.

`budget_remaining` (float, REQUIRED):

Authority budget remaining at `execution_ns`. Non-negative.

`context_drift_pct` (float, REQUIRED):

Context drift percentage from the Scope Authorization Engine.

Range: [0.0, 100.0].

`active_anomalies` (integer, REQUIRED):

Count of active governance anomalies on the delegation chain

at `execution_ns`. Non-negative.

`dr_expires_at` (string, OPTIONAL):

ISO 8601 UTC expiry of the source Delegation Receipt.

`time_remaining_ns` (integer, OPTIONAL):

Nanoseconds until `dr_expires_at`. Zero if expired.

`fragmentation_score` (float, REQUIRED):

Aggregate budget consumption percentage across all active sessions

sharing `chain_root_id`. Range: [0.0, 100.0].

`escalation_event_id` (string, OPTIONAL):

The CEE issued at this RCR, if any. Format: ATFCEE-`{16HEX}`.

`reauth_challenge_id` (string, OPTIONAL):

The RC issued at this RCR, if any. Format: ATFRC-`{16HEX}`.

`sample_reason` (string, REQUIRED):

One of: SCHEDULED, THRESHOLD_BREACH, FRAGMENTATION, EXTERNAL,

RC_TTL_EXPIRED, EXECUTION_COMPLETE.

`content_hash` (string, REQUIRED):

SHA-256 hex digest of the canonical JSON serialization of all

RCR fields except `{content_hash, pqc_signature, pqc_algorithm}`.

See §5.3.

`pqc_signature` (string, OPTIONAL):

Base64-encoded Dilithium-3 (ML-DSA-65) signature over `content_hash`.

`pqc_algorithm` (string, OPTIONAL):

Algorithm identifier. When present, MUST be "dilithium3".

`issued_at` (string, REQUIRED):

ISO 8601 UTC timestamp of RCR issuance. Equals `execution_ts`

for engine-issued RCRs.

`metadata` (object, REQUIRED):

Extension dictionary. MAY be empty.

5.2. Identifier Format

RCR identifiers MUST follow the format:

```
rcr-id = "ATFRCR-" 16HEXDIG
```

where HEXDIG is an uppercase hexadecimal digit (0-9, A-F).

Example: ATFRCR-3F7A2B1C4D5E6F7A

Similarly:

```
cee-id = "ATFCEE-" 16HEXDIG
```

```
rc-id = "ATFRC-" 16HEXDIG
```

5.3. Content Hash Construction

The content hash is computed as:

```
payload = { all RCR fields } - { content_hash, pqc_signature,  
pqc_algorithm }
```

```
serialized = JSON.stringify(payload, sort_keys=True, default=str)
```

```
content_hash = hex(SHA-256(UTF-8(serialized)))
```

Implementations MUST sort all JSON object keys alphabetically before hashing. Floating-point values MUST be serialized with full precision.

5.4. PQC Signature

When a PQC signing provider is available, the engine MUST sign the content_hash using ML-DSA-65 (Dilithium-3, FIPS 204) with the platform private key. The signature MUST be base64-encoded.

If the signing provider is unavailable, the engine MAY issue the RCR with pqc_signature = null and MUST log a WARNING. A content hash is always computed regardless of signature availability.

5.5. Immutability (RGC-INV-005)

An RCR is immutable once issued. Implementations MUST NOT provide an UPDATE path for RCR records. The persistence schema MUST use INSERT ... ON CONFLICT DO NOTHING, never INSERT ... ON CONFLICT DO UPDATE.

6. Continuity Eligibility Score (CES)

6.1. Formula

CES is computed as a weighted sum of four components:

$$CES = (T \times 0.30) + (B \times 0.30) + (D \times 0.20) + (I \times 0.20)$$

CES is a real number in [0.0, 100.0].

6.2. Component Definitions

T – Temporal Health (weight 0.30):

```
If dr_expires_at is null:
```

```
T = 100.0
```

```
Else:
```

```
total_lifetime_ns = dr_expires_at_ns - dr_issued_ns
```

```
remaining_ns = dr_expires_at_ns - now_ns
```

```
If total_lifetime_ns ≤ 0 or remaining_ns ≤ 0:
```

```
T = 0.0
```

```
Else:
```

```
T = max(0.0, min(100.0, remaining_ns / total_lifetime_ns × 100))
```

```
B – Budget Health (weight 0.30):
```

```
If budget_at_admission = 0:
```

```
B = 100.0
```

```
Else:
```

```
B = max(0.0, min(100.0, budget_remaining / budget_at_admission × 100))
```

```
D – Context Fidelity (weight 0.20):
```

```
D = max(0.0, min(100.0, 100.0 - context_drift_pct))
```

```
context_drift_pct is sourced from the Scope Authorization Engine
```

```
(ADR-147). If the scope engine is unavailable, implementations
```

```
SHOULD use context_drift_pct = 0.0 and log a WARNING.
```

```
I – Integrity Score (weight 0.20):
```

```
I = max(0.0, min(100.0, 100.0 - (active_anomalies × 10)))
```

```
Each active governance anomaly on the delegation chain reduces
```

```
I by 10 points. At 10 or more anomalies, I = 0.0.
```

6.3. Threshold Levels

```
CES thresholds and their governance implications:
```



```
■ CES Range ■ Status ■ Required Engine Action ■
```



```
■ 75.0 - 100.0 ■ NOMINAL ■ Continue. Sample at standard interval. ■
```

```
■ 50.0 - 75.0 ■ MONITORING ■ Continue. Increase sampling. Flag outputs. ■
```

```
■ 25.0 - 50.0 ■ WARNING ■ Continue with restrictions. Tier-1 alert. Outputs ■
```

```
■ ■ ■ flagged GOVERNANCE_DEGRADED. ■
```

```
■ 10.0 - 25.0 ■ CRITICAL ■ Suspend new sub-task spawning. Issue RC. Await ■
```

```
■ ■ ■ Tier-1 response within TTL. ■
```

```
■ 0.0 - 10.0 ■ HALT ■ Terminate execution. Issue terminal HALT RCR. ■
```

```
■ ■ ■ Revoke all in-flight sibling sub-tasks. ■
```

Category	Percentage
Top Bar	100%
Bottom Bar	10%

Threshold boundary: NOMINAL begins at $CES \geq 75.0$. A CES of exactly 75.0 is NOMINAL; a CES of 74.999... is MONITORING.

6.4. Computation Requirements (RGC-INV-002)

CES MUST be computed from real-time values at the nanosecond of the RCR. Implementations MUST NOT:

- Cache any CES component across samples.
- Use a CES value computed at a previous sample for the current RCR.
- Apply smoothing, averaging, or interpolation to CES components.

Cached or stale inputs MUST raise InvalidCESInputError.

7. Continuity Chain

7.1. Chain Structure

The continuity chain is a singly linked list of RCRs:

TAR (admission anchor)

```
■■■ RCR (predecessor_rcr_id = null)
```

```

RCR (predecessor_rcr_id = RCR.rcr_id)

```

```

RCR (predecessor_rcr_id = RCR.rcr_id)

```

Page 10

```
■■■ RCR (terminal - sample_reason = EXECUTION_COMPLETE
```

or HALT)

7.2. Acyclicity (RGC-INV-006)

The continuity chain MUST be acyclic. Implementations enforce this by requiring:

For every RCR pair (RCR_i, RCR_j) in a session where $j > i$:

```
RCR█.execution_ns > RCR█.execution_ns
```

```
Implementations MUST detect and reject any attempt to insert an RCR
with execution_ns ≤ the session's last_rcr_ns. If clock skew
produces this condition, the engine MUST advance execution_ns to
last_rcr_ns + 1.
```

7.3. TAR Anchoring (RGC-INV-001)

Every RCR MUST carry a `tar_id` referencing a valid entry in the `atf_temporal_records` table. An RCR with `tar_id = null` or `tar_id` referencing a non-existent record MUST be rejected.

7.4. Chain Verification

To verify a continuity chain, a verifier MUST:

1. Retrieve all RCRs for a session ordered by execution_ns ascending.

2. Verify each RCR's content_hash independently (§5.3).

3. Verify each RCR's pqc_signature where present.

4. Confirm that $\text{RCR}_i.\text{predecessor_rcr_id} = \text{null}$.

5. Confirm that for every pair $(\text{RCR}_i, \text{RCR}_{i+1})$:

$$\text{RCR}_{i+1}.\text{predecessor_rcr_id} = \text{RCR}_i.\text{rcr_id}$$

$$\text{RCR}_{i+1}.\text{execution_ns} > \text{RCR}_i.\text{execution_ns}$$

6. Confirm that every RCR.tar_id is identical (single session anchor).

7. Recompute CES from stored component values and verify consistency

with stored ces_score.

8. Authority Fragmentation Guard (AFG)

8.1. The Fragmentation Attack

Consider a Delegation Receipt DR with authority_budget_granted = 60.

The Monotonic Authority Reduction invariant (RFC-ATF-1 §7.1) prevents any single sub-delegation from exceeding 60. However, an agent may spawn N concurrent sub-agents, each with a grant of $60 / N$.

Individually, each sub-delegation satisfies MAR. Collectively, the sub-agents exercise a combined budget of 60 from a root grant of 60, exhausting the budget across the distribution.

If instead the attacker spawns $(N+k)$ agents, each receiving a grant that individually satisfies MAR, the aggregate budget consumption exceeds the chain root budget by a factor proportional to k. This is the authority fragmentation attack.

RFC-ATF-1 MAR cannot detect this because it operates per-receipt, not per-chain. AFG closes this gap by operating at the chain level.

8.2. AFG Protocol

The AFG MUST be checked before any new sub-delegation is admitted into a Continuity Session:

$$\text{aggregate_consumed} = \sum \text{budget_at_admission}$$

for all ACTIVE sessions on chain_root_id

$$\text{proposed_aggregate} = \text{aggregate_consumed} + \text{new_grant_budget}$$

$$\text{AFG_LIMIT} = \text{chain_root_budget} \times \text{AFG_FRAGMENTATION_LIMIT}$$

If $\text{proposed_aggregate} > \text{AFG_LIMIT}$:

Raise AuthorityFragmentationViolation (RGC-INV-004)

Reject the new sub-delegation

The default AFG_FRAGMENTATION_LIMIT is 0.90 (90%). The remaining 10% serves as a continuity buffer – it cannot be delegated but

provides a residual authority reserve for recovery operations.

Implementations MAY configure AFG_FRAGMENTATION_LIMIT via the AFG_FRAGMENTATION_LIMIT environment variable. Values above 0.95 are NOT RECOMMENDED. Values above 1.0 MUST NOT be accepted.

8.3. Fragmentation Score

Each RCR MUST carry a fragmentation_score field:

$$\text{fragmentation_score} = (\text{aggregate_consumed_at_sample} / \text{total_budget_at_admission}) \times 100$$

A fragmentation_score of 0 indicates no concurrent sub-agents.

A fragmentation_score approaching 90 indicates proximity to the AFG limit and warrants operator attention.

8.4. AFG Invariant (RGC-INV-004)

RGC-INV-004: The aggregate authority budget across all ACTIVE Continuity Sessions sharing a chain_root_id MUST NOT exceed chain_root_budget × AFG_FRAGMENTATION_LIMIT.

Violation raises AuthorityFragmentationViolation. Existing sessions are not terminated by a violation – only the new sub-delegation is rejected.

9. Escalation Protocol

9.1. Continuity Escalation Event (CEE)

A Continuity Escalation Event (CEE) MUST be issued whenever CES transitions to a status other than NOMINAL. The CEE is PQC-signed and persisted to atf_continuity_escalations.

9.2. Threshold-to-Action Mapping

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

MONITORING	WARNING	CRITICAL	HALT	REAUTHORIZE	RC Issued?	Recommended Action	Status
------------	---------	----------	------	-------------	------------	--------------------	--------

9.3. Escalation Event Format

A CEE MUST contain:

cee_id (string): ATFCEE-{{16HEX}}

rcr_id (string): The triggering RCR

```

tar_id (string): Session anchor
delegation_id (string): Source DR
agent_id (string): Executing agent
chain_root_id (string): Chain root
threshold_crossed (string): MONITORING / WARNING / CRITICAL / HALT
recommended_action (string): ALERT / SUSPEND_SPAWNING / REAUTHORIZE / HALT
ces_at_escalation (float): CES value that triggered the CEE
escalation_ns (integer): Nanosecond of escalation
response_ttl_seconds (int): 0 for HALT, implementation-defined for others
content_hash (string): SHA-256 of CEE fields (excl. sig fields)
pgc_signature (string?): Dilithium-3 signature over content_hash
issued_at (string): ISO UTC of CEE issuance

```

10. Reauthorization Challenge (RC)

10.1. Protocol Overview

A Reauthorization Challenge is a formal, cryptographically signed request for mid-execution authority renewal. It is issued exactly when and only when CES drops to CRITICAL ($CES \in [10, 25)$).

The RC is directed to the Tier-1 authority (human operator or Tier-1 agent) that issued the chain root DR. The Tier-1 authority responds by issuing a new, short-lifetime Delegation Receipt bearing the `rc_id` in its metadata.

A single open RC per session is maintained. If CES remains CRITICAL across multiple samples while an unresolved, unexpired RC exists, the engine MUST NOT issue a duplicate RC.

10.2. RC Format

An RC MUST contain:

```

rc_id (string): ATFRC-{16HEX}
cee_id (string): The CEE that triggered this RC
tar_id (string): Session anchor
delegation_id (string): DR being challenged
agent_id (string): Executing agent
chain_root_id (string): Chain root
ces_at_challenge (float): CES at time of RC issuance
challenge_ns (integer): Nanosecond of RC issuance
ttl_seconds (integer): Time allowed for Tier-1 response
expires_at_ns (integer): challenge_ns + (ttl_seconds × 109)
response_dr_id (string?): New DR ID when resolved
resolved (boolean): False until Tier-1 responds or TTL expires

```

```

resolution (string?): "REAUTHORIZED" or "EXPIRED"
content_hash (string): SHA-256 of RC fields (excl. sig fields)
pqc_signature (string?): Dilithium-3 signature over content_hash
issued_at (string): ISO UTC of RC issuance

```

10.3. Challenge-Response Sequence

The complete Reauthorization Challenge protocol proceeds as follows:

```

Step 1: Engine detects  $CES \in [10, 25)$  at sample time  $T_{\blacksquare}$ .
Step 2: Engine issues CEE with threshold_crossed = CRITICAL.
Step 3: Engine issues RC (ATFRC-{16HEX}) with:
    expires_at_ns =  $T_{\blacksquare} + (ttl\_seconds \times 10^{\blacksquare})$ 
Step 4: RC is delivered to the designated Tier-1 escalation endpoint.

```

On successful reauthorization:

```

Step 5: Tier-1 issues new short-lifetime DR (DR') carrying
    metadata.rc_id = rc_id
Step 6: Engine validates DR':
    (a) DR' authority_budget_granted ≤ original DR budget
    (b) DR' issued by a valid Tier-1 principal
    (c) DR' expires_at is after current time
Step 7: If valid, engine resolves RC:
    rc.resolved = true, rc.resolution = "REAUTHORIZED"
Step 8: Engine resets T-component: session.dr_expires_at_ns updated
    to DR'.expires_at_ns
Step 9: Next RCR shows refreshed CES with restored T-component.

```

On TTL expiry without response:

```

Step 5: Engine detects rc.is_expired() = true.
Step 6: Engine issues terminal HALT RCR (§10.5).

```

10.4. TTL Enforcement (RGC-INV-008)

Implementations **MUST** enforce RC TTL with nanosecond-precision comparison:

```
is_expired = (time.time_ns() > rc.expires_at_ns) AND NOT rc.resolved
```

Implementations **SHOULD** check RC TTL at every sample event.

Implementations **MAY** run a background thread that checks TTL at a configurable interval (default: 60 seconds).

10.5. Auto-HALT on Expiry (RGC-INV-003 + RGC-INV-008)

When RC TTL expires without response, the engine **MUST**:

```

1. Issue a terminal RCR with:
    continuity_status = HALT

```

```
sample_reason = RC_TTL_EXPIRED
```

```
metadata.rc_id = rc.rc_id
```

```
metadata.halt_reason = "RC_TTL_EXPIRED"
```

```
2. Set session.status = HALTED.
```

```
3. Issue REVOKED_BY_HALT to all sibling sessions on the same
```

```
chain_root_id (§12, RGC-INV-003).
```

4. Invoke the `halt_callback` if registered.

11. Sampling Strategy

11.1. Execution Profiles

Implementations SHOULD classify executions into profiles:

SHORT: Duration < 60 seconds

MEDIUM: $60 \leq \text{duration} < 600$ seconds

LONG: Duration ≥ 600 seconds

STREAMING: Unbounded duration (no expected termination)

11.2. Sampling Intervals

Recommended sampling intervals by profile and CES status:



■ Profile ■ NOMINAL (s) ■ MONITORING (s) ■ CRITICAL (s) ■



■ SHORT ■ None ■ 15 ■ 5 ■

■ MEDIUM ■ 60 ■ 30 ■ 10 ■

■ LONG ■ 120 ■ 60 ■ 20 ■

■ STREAMING ■ 30 ■ 15 ■ 5 ■



Configurable via RGC_SAMPLE_INTERVAL_SECONDS environment variable.

WARNING status sampling SHOULD use MONITORING intervals.

11.3. Manual Sampling

Any party with access to the engine MAY trigger a manual RCR at any

```
time by calling sample() with sample_reason = EXTERNAL. Manual
```

samples do not reset the automatic sampling clock.

12. RGC Invariants

This section specifies all eight invariants of RFC-ATF-2. Each

```
invariant MUST be enforced by any ATF-RGC-Compliant implementation.
```

RGC-INV-001: TAR Anchoring

Every RCR MUST carry a `tar_id` referencing a valid, non-null

```
Temporal Admissibility Record. Enforcement: tar_id NOT NULL,
```

validated against `atf_temporal_records` at session start.

RGC-INV-002: Real-Time CES Computation

CES MUST be computed exclusively from values current at the nanosecond of the RCR. Cached, averaged, or stale inputs MUST raise `InvalidCESInputError`.

RGC-INV-003: HALT Propagation

When a session enters HALT status, all ACTIVE sessions sharing the same `chain_root_id` MUST receive `status = REVOKED_BY_HALT`. The `halt_callback`, if registered, MUST be invoked.

RGC-INV-004: Authority Fragmentation Guard

The aggregate authority budget across all ACTIVE sessions sharing a `chain_root_id` MUST NOT exceed `chain_root_budget × AFG_LIMIT`. Violation raises `AuthorityFragmentationViolation`.

RGC-INV-005: RCR Immutability

An RCR MUST NOT be modified after issuance. Persistence MUST use `INSERT ... ON CONFLICT DO NOTHING`. No UPDATE path is permitted.

RGC-INV-006: Chain Acyclicity

The continuity chain MUST be acyclic. Each RCR's `execution_ns` MUST be strictly greater than its predecessor's `execution_ns`.

RGC-INV-007: CES Input Freshness

CES inputs MUST not exceed the following staleness thresholds: 5 seconds for CRITICAL sessions, 30 seconds for NOMINAL sessions. Stale inputs MUST raise `InvalidCESInputError`.

RGC-INV-008: RC TTL Enforcement

A Reauthorization Challenge TTL MUST be enforced with nanosecond precision. Auto-HALT MUST be triggered on expiry without response.

All eight invariants extend (but do not replace) the six invariants of RFC-ATF-1. An ATF-RGC-Compliant implementation satisfies all fourteen invariants (ATF-INV-001-006 + RGC-INV-001-008).

13. Wire Format

13.1. Runtime Continuity Record (JSON)

```
{
  "rcr_id": "ATFRCR-3F7A2B1C4D5E6F7A",
  "tar_id": "ATFTAR-1A2B3C4D5E6F7A8B",
  "delegation_id": "ATFDRL-8B2C4D6E1F3A5B7C",
  "agent_id": "AID-FINANCE-3A7F9B2C1D4E5F6A",
  "chain_root_id": "ATFDRL-ROOT0000000000001",
```

```

"execution_ns": 1747180800000000000,
"execution_ts": "2026-05-14T00:00:00.000000+00:00",
"ces_score": 82.5,
"ces_temporal": 90.0,
"ces_budget": 85.0,
"ces_context": 75.0,
"ces_integrity": 70.0,
"continuity_status": "NOMINAL",
"predecessor_rcr_id": "ATFRRCR-2E8D1C9B7A4F3E6D",
"budget_at_admission": 80.0,
"budget_remaining": 68.0,
"context_drift_pct": 25.0,
"active_anomalies": 3,
"dr_expires_at": "2026-05-14T02:00:00.000000+00:00",
"time_remaining_ns": 7200000000000,
"fragmentation_score": 15.0,
"escalation_event_id": null,
"reauth_challenge_id": null,
"sample_reason": "SCHEDULED",
"content_hash": "a3f7c2e1b4d6f8a9...",
"pgc_signature": "base64-dilithium3-signature...",
"pgc_algorithm": "dilithium3",
"issued_at": "2026-05-14T00:00:00.000000+00:00",
"metadata": {}
}

```

13.2. Continuity Eligibility Score (JSON)

```

{
  "ces_score": 82.5,
  "ces_temporal": 90.0,
  "ces_budget": 85.0,
  "ces_context": 75.0,
  "ces_integrity": 70.0,
  "continuity_status": "NOMINAL"
}

```

13.3. Continuity Escalation Event (JSON)

```

{
  "cee_id": "ATFCEE-7B3D5F1A2C4E6F8A",
  "rcr_id": "ATFRRCR-3F7A2B1C4D5E6F7A",
  "tar_id": "ATFTAR-1A2B3C4D5E6F7A8B",
  "delegation_id": "ATFDRCR-8B2C4D6E1F3A5B7C",

```

```
"agent_id": "AID-FINANCE-3A7F9B2C1D4E5F6A",
"chain_root_id": "ATFDR-ROOT000000000001",
"threshold_crossed": "CRITICAL",
"recommended_action": "REAUTHORIZE",
"ces_at_escalation": 18.5,
"escalation_ns": 1747180900000000000,
"response_ttl_seconds": 300,
"resolved": false,
"resolution_status": null,
"content_hash": "b4d6f8a9c2e1b3f7...",
"pgc_signature": "base64-dilithium3-signature...",
"pgc_algorithm": "dilithium3",
"issued_at": "2026-05-14T00:01:40.000000+00:00",
"metadata": {}
}
```

13.4. Reauthorization Challenge (JSON)

```
{
"rc_id": "ATFRC-9C5E7A3B1D2F4A6C",
"cee_id": "ATFCEE-7B3D5F1A2C4E6F8A",
"tar_id": "ATFTAR-1A2B3C4D5E6F7A8B",
"delegation_id": "ATFDR-8B2C4D6E1F3A5B7C",
"agent_id": "AID-FINANCE-3A7F9B2C1D4E5F6A",
"chain_root_id": "ATFDR-ROOT000000000001",
"ces_at_challenge": 18.5,
"challenge_ns": 1747180900000000000,
"ttl_seconds": 300,
"expires_at_ns": 1747181200000000000,
"response_dr_id": null,
"resolved": false,
"resolution": null,
"content_hash": "c5e7a3b1d2f4a6c9...",
"pgc_signature": "base64-dilithium3-signature...",
"pgc_algorithm": "dilithium3",
"issued_at": "2026-05-14T00:01:40.000000+00:00",
"metadata": { "cee_id": "ATFCEE-7B3D5F1A2C4E6F8A" }
}
```

13.5. RCR Verification Request / Response

Request:

```
{
"rcr_id": "ATFRCR-3F7A2B1C4D5E6F7A"
```



```

}
-- or --
{
  "rcr": { <RCR object> }
}

Response:
{
  "rcr_id": "ATFRCR-3F7A2B1C4D5E6F7A",
  "hash_valid": true,
  "pgc_valid": true,
  "pgc_checked": true,
  "chain_position": 3,
  "predecessor_valid": true,
  "ces_recomputed": 82.5,
  "ces_match": true,
  "continuity_status": "NOMINAL",
  "tar_id": "ATFTAR-1A2B3C4D5E6F7A8B",
  "fully_verified": true
}

```

14. Verification Protocol

14.1. Single RCR Verification

To verify a single RCR, a verifier MUST:

1. Recompute content_hash from the RCR payload per §5.3.
2. Assert computed hash = rcr.content_hash.
3. If pgc_signature is present, verify the signature against the platform public key using ML-DSA-65.
4. Recompute CES from stored component values and assert

$$\text{ces_score} = (\text{ces_temporal} \times 0.30) + (\text{ces_budget} \times 0.30) + (\text{ces_context} \times 0.20) + (\text{ces_integrity} \times 0.20).$$
5. Assert continuity_status is consistent with ces_score per §6.3.

14.2. Continuity Chain Verification

A chain verification MUST additionally:

6. Assert $\text{RCR}_i.\text{predecessor_rcr_id} = \text{null}$.
7. For every $(\text{RCR}_i, \text{RCR}_{i+1})$ pair: assert predecessor linkage correct.
8. Assert execution_ns is strictly increasing.
9. Assert all RCRs carry identical tar_id.

14.3. Independent Offline Verification

RFC-ATF-2 preserves ATF-INV-006 (RFC-ATF-1 §7.6): all verification

MUST be performable without platform access. An offline verifier requires only:

- (a) The RCR (or chain) to be verified.
- (b) The platform's Dilithium-3 public key.
- (c) The SHA-256 and ML-DSA-65 implementations.

No API access, no account, no online connectivity required.

15. Persistence Schema

15.1. atf_runtime_continuity

```
CREATE TABLE IF NOT EXISTS atf_runtime_continuity (
```

```
  rcr_id VARCHAR(64) PRIMARY KEY,
```

```
  tar_id VARCHAR(64) NOT NULL,
```

```
  delegation_id VARCHAR(64) NOT NULL,
```

```
  agent_id VARCHAR(64) NOT NULL,
```

```
  chain_root_id VARCHAR(64) NOT NULL,
```

```
  execution_ns BIGINT NOT NULL,
```

```
  execution_ts TIMESTAMPTZ NOT NULL,
```

```
  ces_score FLOAT NOT NULL,
```

```
  ces_temporal FLOAT NOT NULL,
```

```
  ces_budget FLOAT NOT NULL,
```

```
  ces_context FLOAT NOT NULL,
```

```
  ces_integrity FLOAT NOT NULL,
```

```
  continuity_status VARCHAR(16) NOT NULL,
```

```
  predecessor_rcr_id VARCHAR(64) DEFAULT NULL,
```

```
  budget_at_admission FLOAT NOT NULL,
```

```
  budget_remaining FLOAT NOT NULL,
```

```
  context_drift_pct FLOAT NOT NULL DEFAULT 0.0,
```

```
  active_anomalies INT NOT NULL DEFAULT 0,
```

```
  dr_expires_at TIMESTAMPTZ DEFAULT NULL,
```

```
  time_remaining_ns BIGINT DEFAULT NULL,
```

```
  fragmentation_score FLOAT NOT NULL DEFAULT 0.0,
```

```
  escalation_event_id VARCHAR(64) DEFAULT NULL,
```

```
  reauth_challenge_id VARCHAR(64) DEFAULT NULL,
```

```
  sample_reason VARCHAR(32) NOT NULL DEFAULT 'SCHEDULED',
```

```
  content_hash VARCHAR(64) NOT NULL,
```

```
  pqc_signature TEXT DEFAULT NULL,
```

```
  pqc_algorithm VARCHAR(32) DEFAULT NULL,
```

```
  issued_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
  metadata JSONB NOT NULL DEFAULT '{}'
```

```
);
```

Required indices:

```
CREATE INDEX IF NOT EXISTS idx_atf_rcr_tar
ON atf_runtime_continuity (tar_id);

CREATE INDEX IF NOT EXISTS idx_atf_rcr_delegation_status
ON atf_runtime_continuity (delegation_id, continuity_status);

CREATE INDEX IF NOT EXISTS idx_atf_rcr_chain_root_ns
ON atf_runtime_continuity (chain_root_id, execution_ns);

CREATE INDEX IF NOT EXISTS idx_atf_rcr_status_issued
ON atf_runtime_continuity (continuity_status, issued_at);

CREATE INDEX IF NOT EXISTS idx_atf_rcr_agent_ns
ON atf_runtime_continuity (agent_id, execution_ns);
```

15.2. atf_continuity_escalations

```
CREATE TABLE IF NOT EXISTS atf_continuity_escalations (
  cee_id VARCHAR(64) PRIMARY KEY,
  rcr_id VARCHAR(64) NOT NULL,
  tar_id VARCHAR(64) NOT NULL,
  delegation_id VARCHAR(64) NOT NULL,
  agent_id VARCHAR(64) NOT NULL,
  chain_root_id VARCHAR(64) NOT NULL,
  threshold_crossed VARCHAR(16) NOT NULL,
  recommended_action VARCHAR(32) NOT NULL,
  ces_at_escalation FLOAT NOT NULL,
  escalation_ns BIGINT NOT NULL,
  response_ttl_seconds INT NOT NULL DEFAULT 300,
  response_received_at TIMESTAMPTZ DEFAULT NULL,
  response_dr_id VARCHAR(64) DEFAULT NULL,
  resolved BOOLEAN NOT NULL DEFAULT FALSE,
  resolution_status VARCHAR(32) DEFAULT NULL,
  content_hash VARCHAR(64) NOT NULL,
  pqc_signature TEXT DEFAULT NULL,
  pqc_algorithm VARCHAR(32) DEFAULT NULL,
  issued_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  metadata JSONB NOT NULL DEFAULT '{}';
);
```

Required indices:

```
CREATE INDEX IF NOT EXISTS idx_atf_cee_tar
ON atf_continuity_escalations (tar_id);

CREATE INDEX IF NOT EXISTS idx_atf_cee_chain_root
ON atf_continuity_escalations (chain_root_id, resolved);
```

16. API Endpoints

The following endpoints MUST be implemented by an ATF-RGC-Compliant server:

Method	Path	Description
POST	/api/atf/continuity/start	Start a continuity session (TAR-anchored). Returns session state.
POST	/api/atf/continuity/sample	Emit a manual RCR for an active session. Returns RCR + CES.
GET	/api/atf/continuity/{rcr_id}	Retrieve a single RCR by ID.
GET	/api/atf/continuity/session/{tar_id}	Retrieve the full continuity chain for an execution, ordered by ns.
GET	/api/atf/continuity/health/{delegation_id}	Current CES for an active DR. Non-destructive (no RCR emitted).
POST	/api/atf/continuity/reauthorize	Submit RC response (new DR). Resolves challenge, resets T-CES.
GET	/api/atf/continuity/sessions	List all active continuity sessions.
GET	/api/atf/escalations/{chain_root_id}	All CEEs for a chain root.
POST	/api/atf/continuity/fragmentation/check	Validate a proposed sub-delegation against the AFG. Returns allowed/denied + fragmentation score.

17. Security Considerations

17.1. CES Manipulation

An attacker with control over CES inputs may attempt to keep CES artificially high to avoid escalation. Mitigations:

- (a) Context drift input MUST come from an independent Scope Authorization Engine, not from the executing agent itself.
- (b) Active anomaly count MUST be sourced from an independent governance monitoring layer.
- (c) Temporal and budget components are computed by the engine from session state – not from agent-reported values.
- (d) RGC-INV-002 prohibits cached inputs.

17.2. Chain Replay Attack

An attacker may attempt to replay an old, high-CES RCR to misrepresent the current authority health of an execution. Mitigations:

- (a) Each RCR carries execution_ns – an external verifier can detect temporal anomalies.
- (b) The continuity chain is acyclic (RGC-INV-006) – inserted RCRs would break the chain linkage.
- (c) PQC signature verification detects tampered or replayed RCRs.

17.3. Fragmentation Attack

Addressed by the AFG protocol (§8). Implementations MUST enforce the AFG check before every new sub-delegation.

17.4. Clock Drift Attack

An attacker who can manipulate the system clock may produce RCRs with artificially high T-component values. Implementations SHOULD use a monotonic clock source for execution_ns. Implementations MAY additionally validate execution_ns against an external time authority.

17.5. RC Forgery

An attacker may attempt to forge an RC response by submitting a DR that does not originate from a valid Tier-1 principal. The engine MUST validate the response DR against the Trust Lattice before accepting a reauthorization.

17.6. Phantom HALT

An attacker with write access to the anomaly count (active_anomalies) input may manufacture artificial HALTs by inflating the anomaly count. The anomaly source MUST be access-controlled and sourced from the independent governance monitoring layer.

17.7. Budget Inflation Attack

An attacker may attempt to report a higher `budget_remaining` than actual to keep B-component high. The engine computes `budget_remaining` from its own session state (updated by `budget_consumed` inputs at each sample), not from agent-reported values.

18. Compliance Mapping

■ Regulation / Standard ■ Gap Closed by RFC-ATF-2 ■

■ EU AI Act Art. 13 ■ Continuous evidence of authorized scope throughout ■

■ (Transparency) ■ execution, not only at admission. ■

■ EU AI Act Art. 9 ■ Risk management extends to runtime authority degradation, ■

■ (Risk Management) ■ not only design-time assessment. ■

■ DORA Art. 9 ■ ICT operational control evidence for the full duration of ■

■ (ICT Risk Management) ■ autonomous actions, satisfying continuous monitoring. ■

■ MiCA Recital 65 ■ Algorithmic trading: CES provides continuous position ■

■ (Algorithmic Trading) ■ authority validation against market manipulation vectors. ■

■ SOC 2 CC6 ■ Logical access controls enforced and evidenced throughout ■

■ (Logical Access) ■ execution lifetime, not only at session initiation. ■

■ ISO 27001 A.9.4 ■ Information access control: runtime enforcement, not just ■

■ (System Access Control) ■ admission-time. ■

■ NIST AI RMF Govern 1.4 ■ Continuous monitoring of AI system behavior within ■

■ (Continuous Monitoring) ■ authorized parameters with cryptographic evidence. ■

■ OSFI E-23 ■ Model risk management: runtime authority health provides ■

■ (Canada Banking Regulator) ■ continuous evidence trail for AI governance audits. ■

19. Extension Points

Implementations MAY extend RFC-ATF-2 with the following:

RCR → W3C Verifiable Credential Export:

Export RCR chains as W3C Verifiable Presentations for cross-organizational governance verification without ATF infrastructure.

Predictive CES:

An ML model trained on historical CES trajectories to predict CRITICAL events 60-120 seconds before threshold crossing, enabling proactive reauthorization.

Multi-Instance Synchronization:

Synchronize the fragmentation guard across horizontally scaled instances via a distributed state broadcast (e.g., Redis pub/sub), ensuring AFG accuracy in multi-replica deployments.

RCR Streaming:

Real-time streaming of RCRs to external SIEM or governance monitoring platforms via webhook or message queue.

Cross-Organization Continuity Verification:

Protocol for verifying continuity chains across organizational boundaries where the verifying party does not have access to the issuing platform's internal state.

20. Relationship to RFC-ATF-1

RFC-ATF-2 is a strict extension of RFC-ATF-1. All RFC-ATF-1 artifacts (AIR, DR, Trust Lattice, TAR) remain normative and unchanged. RFC-ATF-2 adds Layer 4 without modifying Layers 1-3.

Compliance levels:

ATF-Compliant:

Implements RFC-ATF-1 in full (all six invariants, DR issuance, PQC signing, offline verification).

ATF-Temporal-Compliant:

Additionally implements Temporal Authority Admissibility (ADR-157): TAR issuance, nanosecond-precise admission proofs.

ATF-RGC-Compliant:

Additionally implements RFC-ATF-2 in full (all eight RGC invariants, RCR issuance, CES computation, AFG, escalation protocol, RC protocol, offline chain verification).

An implementation that is ATF-RGC-Compliant satisfies all fourteen invariants across both documents and provides full lifecycle governance

coverage from agent registration through execution completion.

21. References

[RFC-ATF-1]

Nunes, H., "RFC-ATF-1: Agent Trust Fabric Delegation Protocol, Version 1.0.0", OMNIX QUANTUM Open Standard, May 2026.

DOI: 10.5281/zenodo.20155016

SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6757339

[FIPS204]

National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard (ML-DSA)", FIPS 204, August 2024.

[RFC2119]

Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[RFC8174]

Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, May 2017.

[ADR-147]

Nunes, H., "Scope Authorization Record", OMNIX ADR-147, 2026.

omnixquantum.net/docs/adr/ADR-147

[ADR-156]

Nunes, H., "Agent Trust Fabric", OMNIX ADR-156, 2026.

omnixquantum.net/docs/adr/ADR-156

[ADR-157]

Nunes, H., "Temporal Authority Admissibility", OMNIX ADR-157, 2026.

omnixquantum.net/docs/adr/ADR-157

[ADR-158]

Nunes, H., "Cross-Domain Trust Portability", OMNIX ADR-158, 2026.

omnixquantum.net/docs/adr/ADR-158

[ADR-159]

Nunes, H., "Runtime Governance Continuity", OMNIX ADR-159, 2026.

omnixquantum.net/docs/adr/ADR-159

[TLA-ATF]

Nunes, H., "ATF-TLA-SPEC: Formal TLA+ Specification of ATF Invariants", OMNIX QUANTUM, May 2026.

DOI: 10.5281/zenodo.20155016 (included in v1.0.0 archive)

22. Appendix A – CES Computation Examples

A.1. NOMINAL Session – No Expiry, Full Budget

Inputs:

`dr_expires_at = null`

`budget_remaining = 80.0`

`budget_admission = 80.0`

`context_drift_pct = 0.0`

`active_anomalies = 0`

$T = 100.0$

$B = 80.0 / 80.0 \times 100 = 100.0$

$D = 100.0 - 0.0 = 100.0$

$I = 100.0 - 0 \times 10 = 100.0$

$CES = (100.0 \times 0.30) + (100.0 \times 0.30) + (100.0 \times 0.20) + (100.0 \times 0.20)$

$= 30.0 + 30.0 + 20.0 + 20.0 = 100.0$ – NOMINAL

A.2. CRITICAL Session – DR Nearing Expiry, Budget Drained

Inputs:

`time_remaining_ns = 300_000_000_000 (5 minutes)`

`total_lifetime_ns = 7_200_000_000_000 (2 hours)`

`budget_remaining = 8.0`

`budget_admission = 80.0`

`context_drift_pct = 30.0`

`active_anomalies = 2`

$T = 300_000_000_000 / 7_200_000_000_000 \times 100 = 4.17$

$B = 8.0 / 80.0 \times 100 = 10.0$

$D = 100.0 - 30.0 = 70.0$

$I = 100.0 - 2 \times 10 = 80.0$

$CES = (4.17 \times 0.30) + (10.0 \times 0.30) + (70.0 \times 0.20) + (80.0 \times 0.20)$

$= 1.25 + 3.0 + 14.0 + 16.0 = 34.25$ – WARNING

(RC issued only at CRITICAL, i.e. $CES < 25$)

A.3. HALT – Complete Budget Exhaustion and Context Collapse

Inputs:

`T = 0.0 (DR expired)`

`B = 0.0 (budget exhausted)`

`D = 0.0 (100% context drift)`

`I = 0.0 (10+ anomalies)`

`CES = 0.0 – HALT`

23. Appendix B – RGC Compliance Checklist

Implementations MUST satisfy all items:

- RCR issued at each sample event with all required fields
- `rcr_id` follows `ATFRCR-{16HEX}` format
- `tar_id` is non-null and references a valid TAR (RGC-INV-001)
- CES computed from real-time values only (RGC-INV-002)
- CES formula: $(T \times 0.30) + (B \times 0.30) + (D \times 0.20) + (I \times 0.20)$
- CES thresholds correctly mapped to NOMINAL/MONITORING/WARNING/CRITICAL/HALT
- `content_hash` computed from canonical JSON, `sort_keys=True` (§5.3)
- `pgc_signature` computed with ML-DSA-65 when provider available
- RCR stored with `INSERT ... ON CONFLICT DO NOTHING` (RGC-INV-005)
- Continuity chain linked via `predecessor_rcr_id` (§7.1)
- `execution_ns` strictly increasing within session (RGC-INV-006)
- AFG checked before each new sub-delegation (RGC-INV-004)
- CEE issued on every non-NOMINAL threshold transition (§9)
- RC issued when and only when $CES \in [10, 25)$ (§10)
- RC TTL enforced with nanosecond precision (RGC-INV-008)
- Auto-HALT on RC TTL expiry (RGC-INV-003 + RGC-INV-008)
- HALT revokes all ACTIVE sibling sessions on chain root (RGC-INV-003)
- All artifacts independently verifiable offline (ATF-INV-006)
- All nine API endpoints implemented (§16)

24. Appendix C – Implementation Notes

Reference implementation: `omnix_core/agents/atf/runtime_continuity.py`

Test suite: `tests/test_runtime_governance_continuity.py` (82 tests)

Verification status: 82/82 passing as of May 13, 2026

The reference implementation is structured as a process-level singleton (`get_rgc_engine()`) to ensure a single, consistent fragmentation guard state within a process. Deployments with multiple processes MUST implement a distributed fragmentation guard (see §19, Extension Points).

The reference implementation writes to the database via daemon threads (non-blocking on the execution path). The execution path is never blocked by DB write failures. DB failures are logged as WARNING.

Persistence failures do NOT affect in-memory session state. RCRs are stored in both the in-memory store and the database. The in-memory store is the authoritative source for active sessions.

Author's Address

Harold Nunes (Editor)

OMNIX QUANTUM LTD

United Kingdom

Email: standards@omnixquantum.com

Acknowledgement of formal gap identification:

Akhilesh Warik, Runtime AI Governance Research, May 2026.